Product Description

The growing reliance on wireless networks, particularly in enterprise environments, makes network performance, coverage, and security vital. Administrators face challenges managing traffic, identifying dead zones, and preventing unauthorized access. Traditional solutions often lack robust security features. Our Linux-based WiFi heat mapping software addresses this by combining network coverage visualization with security threat detection, targeting issues like rogue access points and evil twin attacks.

Problem Statement

Current WiFi tools often separate performance monitoring and security threat detection, overlooking threats like unauthorized access points, MAC spoofing, and man-in-the-middle attacks. Existing tools focus on metrics like RSSI without integrating real-time security monitoring. Our product bridges this gap, offering a unified platform for visualizing network coverage and detecting cybersecurity threats.

Why is the Product Necessary?

With the rise of wireless technology in sectors like finance and healthcare, and the expansion of remote work, ensuring secure and efficient WiFi networks is critical. Vulnerabilities from BYOD policies and inadequate coverage analysis lead to breaches. Our tool integrates heat mapping and threat detection, providing a comprehensive solution for organizations needing real-time insights into coverage and security.

Main Stakeholders

- **Network Administrators**: Gain real-time insights into coverage and vulnerabilities.
- **Cybersecurity Teams**: Monitor and address malicious activities, such as rogue access points.
- **IT Managers**: Use actionable data to make informed decisions on infrastructure.
- **Enterprise Users**: Benefit from optimized network coverage and enhanced security.

Product Details

Our Linux-based software offers integrated performance monitoring and security threat detection. Key features include:

- **RSSI Mapping**: Real-time signal strength visualization.
- **Network Load Monitoring**: Identify bandwidth bottlenecks.
- **Security Threat Detection**: Detect rogue access points and spoofed networks.

- **Trilateration**: Pinpoint access point locations via signal strength.

How Does Our Product Solve the Problem?

Unlike traditional tools that focus solely on performance, our product provides a dual approach. It visualizes network coverage gaps and actively detects security threats, enabling preemptive action against vulnerabilities. This integration of performance and security makes it ideal for enterprise networks where efficiency and safety are paramount.

Requirement Specification

1. **Data Collection**: Create scripts for real-time WiFi data collection to identify and resolve network issues.
2. **Heatmap/Loadmap Generation**: Develop algorithms for dynamic visualizations of signal strength and traffic load to diagnose network faults.
3. **Security Threat Detection**: Implement tools for identifying rogue access points, MAC spoofing, and MITM attacks in real time.
4. **Multi-Channel Data Collection**: Enable multi-threaded WiFi channel monitoring for network optimization.
5. **Real-Time Updates**: Provide systems for instant heatmap updates for rapid issue response.

Usability Requirements

1. **User-Friendly Interface**: Simple UI for accessibility across skill levels.
2. **Real-Time Interaction**: Enable dynamic updates for immediate analysis.
3. **Multilingual Compatibility**: Support languages like English and Chinese.
4. **Open-Source**: Free and customizable for broader accessibility.

Technical Requirements

1. **Integration**: Use Python for seamless data collection, analysis, and visualization.
2. **Security**: Ensure secure access and encrypted data handling.
3. **Platform Compatibility**: Support Windows and Linux systems.
4. **Interfaces**: Develop backend APIs and command-line support.

Dependencies

1. **Technical**: WiFi adapters in monitor mode; Python services like netsh and iwconfig.
2. **Regulatory**: Adherence to wireless monitoring and data privacy laws.
3. **Access/Rights**: Requires administrative credentials for network configurations.

Assumptions and Constraints

- **Assumptions**: Availability of WiFi adapters, Linux/Python proficiency, access to open-source libraries, stable environments.
- **Constraints**: Limited hardware, compatibility challenges, time constraints, regulatory compliance needs.

Impact on Development

- Delayed testing due to hardware unavailability.
- Compatibility issues led to partial reliance on Windows.
- Multilingual support diverted resources from other features.
- Time constraints focused efforts on essential functions.

Tools and Resources

1. **Software**: Python, NumPy, Regex, Tkinter.
2. **Hardware**: WiFi adapters, access points, development systems.
3. **Development Tools**: Python IDEs, debugging utilities, validation tools.
4. **Coordination**: Trello, Microsoft Teams, Google Docs.
5. **Data Sources**: RSSI data, predefined coordinates, signal-to-distance conversions.
6. **Modeling Methods**: Trilateration, signal-to-distance conversion, error handling.

Deliverables

A user-friendly tool integrating RSSI analysis, WLAN traffic monitoring, and trilateration visualization with a Tkinter-based GUI. Python algorithms drive data collection, traffic analysis, and coordinate calculation, visualized dynamically in 2D plots. Comprehensive documentation and a demonstration video are included for ease of use and understanding.

Metrics

Criteria	Functionality	Usability	Accuracy	Integration	Documentation	Performance	Stakeholder Approval
Metric	Executes core features seamlessly.	Intuitive and responsive GUI	Precise trilateration calculations.	Seamless GUI-backend interaction.	Clear, comprehensive guides provided.	Quick analysis and results display.	Meets stakeholder requirements.
Success	95% accuracy across scenarios.	90% positive user feedback	±5% margin of error.	Zero functional disconnects	Users independently operate tool.	Meets stress test benchmarks.	Formal approval after the video demo.

Product Development Processes

Team Members and Their Experience and Roles

The team consisted of six members who were all from the field of cyber security with various experiences and skills which were essential for the allocation of roles.

Team Member Name	Experience	Roles
Marco	Networking and programming knowledge including Java and Python.	Team Leader, Coder, Data Visualization
Alexandra	Background in cyber security, has had previous linux courses from other classes throughout education.	Team coordinator, majority report writer, editor,
Zijie	Experience in Python programming, a focus on network analysis. Worked with both Linux and Windows environments.	Coder, Tester, UI Developer.
Nileththi	Background in Cybersecurity and experience in UI design with Python's Tkinter and skills in creating presentations and graphic designs	UI designer, Graphic Designer, Slides Creator.
Modarage	IT backgrounded in cybersecurity,had experience with IT networking and level1 support	Video creator

Meeting Arrangements

Our team arranged weekly meetings, either face-to-face or online via Microsoft Teams, to discuss individual and product progress. If any team members were absent, we used the Teams group chat to update them on key discussions, completed tasks, and upcoming deadlines. After receiving feedback, we started using meeting minutes with a table to track attendance, meeting location, and summarize the agenda. For communication with our product owner, we aimed to meet at least once every two weeks, transitioning from email to Microsoft Teams for more efficient communication through both meetings and group chats.

Development Process

The development of the Linux-based Wi-Fi Heatmap and Load Analyzer began by identifying user needs outlined by our product owner, Justin Lipman. The key requirements included:

- Visual representation of Wi-Fi signal strength across an area.
- Analysis of RF environment/spectrum congestion across an area.
- Real-time overlay of heatmap/loadmap data on a map.

The tool was designed for RF Network and Cybersecurity Engineers to aid in deploying and analyzing wireless networks. A modular design approach was adopted to enhance efficiency and flexibility, allowing updates to individual features without impacting the entire system. Figure 1, created using Microsoft Visio, illustrates the tool's functionality.

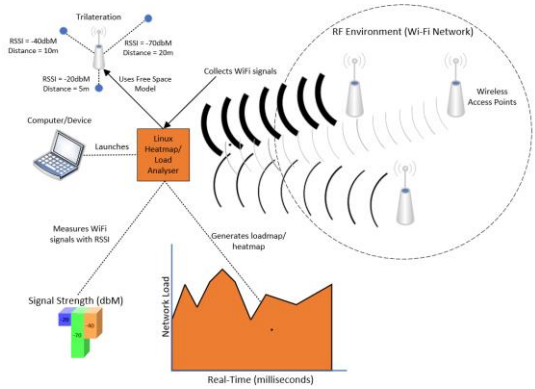

Figure 1: Basic visual diagram of the Linux WiFi Heatmap and Load Analyzer functionality where the tool collects WiFi signals from various networks within an RF environment and measures that signal strength, generates loadmaps/heatmaps, and performs trilateration.

Modifying the Code Provided by Justin

In the early stages of the project, Justin provided us with starter code that we modified to suit our needs. This was our first hands-on experience with the code and served as a valuable introduction to the project. Through this process, we recognized the complexity of the requirements and the necessity of adapting the code for different environments. It also highlighted the importance of understanding the original intent of the code, as even minor changes could lead to unexpected issues if the underlying purpose was not fully grasped.

Configuring Linux Environment

Configuring the Linux environment to run the code was challenging, particularly with Wi-Fi operations like setting the network interface to monitor mode. We encountered various issues, including driver compatibility and permission errors, which required a deeper understanding of Linux networking tools. Despite not fully configuring the environment, this process taught us the importance of troubleshooting across software and hardware levels, patience in addressing technical challenges, and how Linux interacts with network interfaces. It reinforced the value of persistence in problem-solving.

Switching Back to Windows System

After prolonged challenges with the Linux setup, we switched to Windows to keep the project on track. Although it felt like a setback, the decision proved productive as we made progress using Windows-specific commands like netsh. However, the transition brought unexpected issues, such as command outputs being in Mandarin due to a system language setting, complicating data extraction. This taught us the importance of anticipating and adapting to such challenges. The experience underscored the need for flexibility in software development and the willingness to pivot when necessary to move forward.

Developing the Core Functionality

We began by writing a Python script to execute the command `netsh wlan show network mode=Bssid` (output shown in Figure 2) to extract the signal strength of each wireless connection and convert it from percentage to dBm. While the concept was straightforward, implementing it was challenging. The primary issue was unreliable data extraction, which we eventually traced to Mandarin output from one team member's system. Adding language filtering to handle both English and Mandarin resolved the problem (as shown in Figure 3).

This experience emphasized the importance of attention to detail and adaptability when encountering unexpected challenges. It also highlighted how seemingly minor factors, like language settings, can significantly affect project outcomes.

```
SSID 6 : Kogan_A6AA_5G
    Network type            : Infrastructure
    Authentication          : WPA2-Personal
    Encryption              : CCMP
    BSSID 1                 : c4:71:54:f7:a6:a9
         Signal             : 85%
         Radio type         : 802.11ac
         Band               : 5 GHz
         Channel            : 36
         Basic rates (Mbps) : 6 12 24
         Other rates (Mbps) : 9 18 36 48 54

SSID 7 : Kogan_A6AA_5G_5GEXT
    Network type            : Infrastructure
    Authentication          : WPA2-Personal
    Encryption              : CCMP
    BSSID 1                 : 94:a6:7e:2c:3c:f2
         Signal             : 24%
         Radio type         : 802.11ac
         Band               : 5 GHz
         Channel            : 36
         Basic rates (Mbps) : 6 12 24
         Other rates (Mbps) : 9 18 36 48 54
```

Figure 2: Use of the ‘netsh wlan show networks mode=bssid' command in a Windows CMD terminal to perform a scan of all nearby WiFi networks.

```
SSID 1 : UTS-WiFi
    Network type            : Infrastructure
    Authentication          : WPA2-Enterprise
    Encryption              : CCMP
    BSSID 1                 : 90:4c:81:a2:bc:d2
         Signal             : 90%
         Radio type         : 802.11n
         Band               : 5 GHz
         Channel            : 132
         Basic rates (Mbps) : 12 24 54
         Other rates (Mbps) : 18 36 48


Extracted SSID & MAC Address: ['UTS-WiFi', '90:4c:81:a2:bc:d2']
Extracted RSSI Values (%): [90]
Filtered SSIDs: ['UTS-WiFi']
SSID to RSSI Mapping: {'UTS-WiFi': [90]}
Network: UTS-WiFi, RSSI: -5 dBm
```

Figure 3: First successfully working iteration of the tool that runs on a Python script and uses the ‘netsh wlan show networks mode=bssid' command to collect information about all nearby WiFi networks and then converts the given signal strength percentage to a dBm value.

```
WiFi Network Analysis Tool
1. Network Analysis
2. Channel Load
3. WLAN Connection Traffic Analysis
4. Trilateration (Coming Soon)
5. Exit
Enter your choice (In Number): 5
Exiting... Goodbye!
```

Figure 3: The first iteration of the user interface for the tool which offers various network utility options including

Combining Code with the User Interface

Towards the end of the project, we integrated our Python code with the user interface developed by team members (Figure 4). This presented new challenges as we had to consider usability and how the tool would feel to users. It was no longer just about accurate calculations but ensuring our code fit into the larger system. The process involved extensive collaboration, requiring clear communication to align with the team's work. Multiple integration tests were necessary to ensure seamless functionality across components. This experience highlighted the importance of understanding the bigger picture, focusing beyond individual tasks, and the satisfaction of contributing to a cohesive final product.

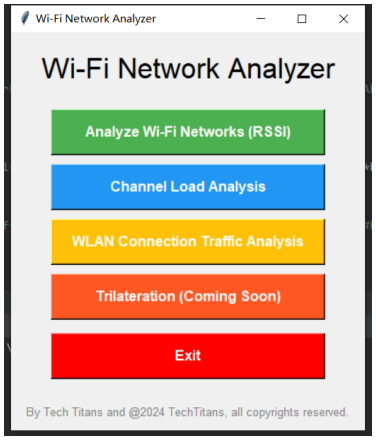

Figure 4: A newly designed interactive graphical user interface which was selected and integrated with the Python script to create a basic working prototype of the tool.

Figures 5-7 demonstrates how the tool functions for each of the options (except for the trilateration option as it was out of the scope of the product to integrate this functionality due to the complexity).

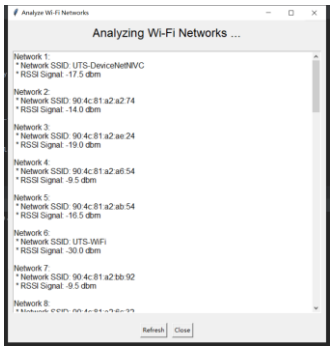

Figure 5: New UI to display output of the WiFi networks scanning option for the tool.

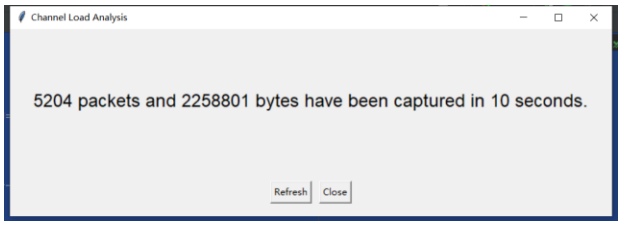

Figure 6: The channel load analysis option for the tool that records the amount of packets and bytes that were captured by the wireless network interface within the last 10 seconds.

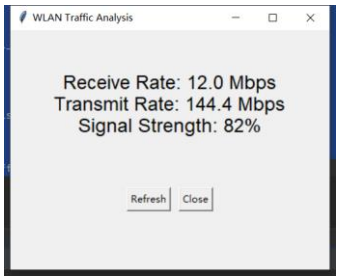

Figure 7: The output window for the WLAN connection traffic analysis option which records the transmission and receive rate of the data packets per second along with the signal strength of the WiFi connection.

Heatmaps/Loadmaps

After getting all the WiFi utility options working for the tool, the next goal was to develop the heatmap/loadmaps functionality. Before commencing the development process, we selected a modular approach for the product which proved to be beneficial as it allowed us to divide both the functionalities of collecting WiFi signals from within an RF environment and the generation of heatmaps/loadmaps into two separate features rather than dependent features. Unfortunately due to time constraints and complexity, we were unable to combine the features of the WiFi collection signals and heatmap/loadmap generation so we developed both features as separate modules. With guidance from Justin, we were able to develop a basic sample of a WiFi heatmap/loadmap as a starting point (as seen in Figure 8 below). After reviewing the initial design of the heatmap/loadmap, Justin provided further feedback to more accurately assess the RF environment by gridding the floor plan and using the prototype to measure the WiFi signal strength of each specific position of the house (as seen in Figure 9 below).

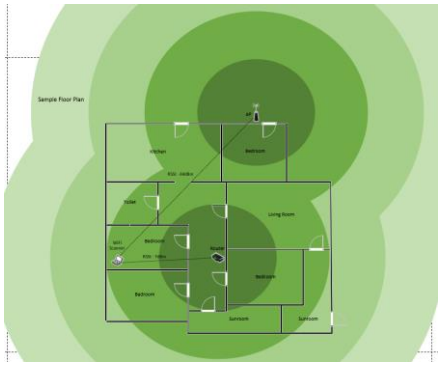

Figure 8: First iteration of a heatmap/loadmap of one of the member's house floor plan to visually represent the strength of WiFi signals within this RF environment. (Created using Microsoft Visio)

Figure 9: Second iteration of the WiFi heatmap/loadmap of the same floor plan where one of the members used a grid and the prototype to accurately assess the overall WiFi signal strength coverage across their house which involved recording the average signal strength of each area within that grid. (Colour-coded with dark green being the strongest signal and light red being the weakest signal)

Trilateration

The final feature Justin proposed was trilateration, a method to determine the exact location of a wireless access point by measuring distances from at least three known points, similar to GPS technology. After discussions, we decided trilateration was beyond the prototype's scope due to time and complexity constraints. However, we created a separate working prototype using the Free Space model formula, which calculates distances based on WiFi signal strength (RSSI). The formula was calibrated to the floor plan, with a grid system where each cell represented one meter, and three known points were mapped. We

developed a Python script to input grid coordinates and estimate the access point's location, as shown in Figures 10 and 11.

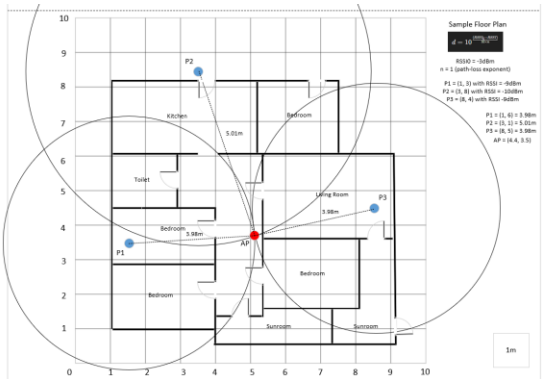

Figure 10: Basic visual diagram of trilateration by using a simple grid coordinate system to map out three known locations and their distance from the access point by substituting their respective RSSI values into the FreeSpace formula. The distance and grid coordinate values are then substituted into the trilateration Python prototype to calculate the rough location of the unknown access point.

```
ou extract a single element from your array before performing this operation. (Deprecated NumPy 1.25.)
  return (float(x), float(y))
Calculated location of unknown access point: (4.488783333333333, 3.5785166666666672)
```

Figure 11: Output of the trilateration Python prototype code that takes the coordinate and distance values of the three known locations and then generates the location of the unknown access point as grid coordinates (fairly accurate when compared with the access point location that is mapped in Figure 10).

Task Allocation and Timeline

Team members	AL	MZ	NS	ZL	MP	Total
Background research	16	18	15	18	10	77
Team leadership/coordination	20	20	10	10	18	78
Software Development	0	4	8	45	0	57
Report Structure	12	8	6	5	5	36

Video Creation	0	0	0	0	14	14
Presentation preparation and review	8	10	20	10	10	58
Total	56	60	59	88	57	

Breakdown of Development Schedule with Major Milestones

Alex

Product specific INDIVIDUAL Milestones	% done				
	W4	W6	W8	W10	W12
W4 Milestone Complete Initial Research	80%	100%	100%	100%	100%
W6 Milestone Design visual concepts for heat mapping and potential cybersecurity vulnerabilities.	0%	30%	80%	90%	100%
W8 Milestone Refining report sections, adding in technical details about the product and its impact on network security	0%	10%	60%	80%	100%
W10 Milestone Finalize the roadshow presentation, ensuring it communicates the product's key technical and business benefits.	0%	0%	40%	80%	100%

W12 Milestone Document, Refine Report, and Prepare Final Presentation	0%	0%	60%	80%	90%

Marco

Product specific INDIVIDUAL Milestones	% done				
	W4	W6	W8	W10	W12
W4 Milestone - Complete product research	50%	75%	100%	100%	100%
W6 Milestone - Complete the design of the product functionality and plot it in a diagram for the roadshow	20%	100%	100%	100%	100%
W8 Milestone - Finalize the development of the product code and selecting the optimal environment. Keep tracking about the	0%	10%	50%	75%	100%
W10 Milestone - Checking with the product owner to ensure all product functionalities and features meet requirements and expectations	10%	25%	55%	85%	100%
W12 Milestone - Finalize product code, data visualization and all other necessary product functionalities to be assembled in the presentation for the final roadshow	5%	20%	45%	65%	90%

Zijie

Product specific INDIVIDUAL Milestones	% done				
	W4	W6	W8	W10	W12
W4 Milestone - Initial setup and configuration (Linux and Windows environments)	50%	80%	100%	100%	100%
W6 Milestone - Develop Python scripts for Wi-Fi signal analysis (RSSI extraction)	20%	60%	100%	100%	100%
W8 Milestone - Add network throughput and channel load features	0%	20%	50%	80%	100%
W10 Milestone - Integrate Python scripts with the Tkinter user interface	0%	10%	40%	80%	100%
W12 Milestone - Test and debug tool in multiple environments, finalise for presentation	0%	10%	30%	70%	90%

Nileththi

Product specific INDIVIDUAL Milestones	% done				
	W4	W6	W8	W10	W12
W4 Milestone - Product research completion	60%	80%	100%	100%	100%
W6 Milestone - Finalize the security features to be integrated into the product and complete the flyers and graphic designs for the roadshow.	30%	85%	100%	100%	100%
W8 Milestone - Designing a visual representation to the application.	0%	10%	75%	100%	100%
W10 Milestone - Designing and finalising presentation slides, graphics, and pod decorations for the roadshow.	0%	0%	20%	100%	100%
W12 Milestone - Finalise the ui design code with Ziije, and completion of the product requirement specification section in the report.	0%	0%	30%	65%	100%

Validation and Testing

The validation and testing processes for the WiFi Heatmap and Load Analyzer were used to ensure that the product meets functional, performance and usability requirements. These processes were conducted throughout the development stages and were essential for maintaining product integrity and aligning with our product owner’s expectations.

Outline of Testing and Validation Processes

The testing phase was one of the most intense and eye-opening parts of the project. It wasn't just about running the script once or twice to see if it worked—it was about systematically trying to break the code and identifying all the ways it could fail, then figuring out how to make it more robust. Here are the key steps and our reflections during each part of the testing process:

Initial Testing on Linux

Setting the network interface to monitor mode on Linux proved challenging due to driver incompatibilities and permissions issues. Despite hours of research, configuration tweaks, and trying various Linux distributions, progress felt elusive. A major obstacle was the varying capabilities of Wi-Fi adapters—some supported monitor mode, while others did not, a lesson we learned the hard way. After numerous failed attempts to configure the virtual machine, we decided, after discussions with Justin and the team, to switch to Windows to keep the project on track.

This experience highlighted the limitations of software when hardware isn’t compatible and taught us the importance of knowing when to pivot instead of persisting with an unworkable approach. It was a valuable lesson in balancing determination with practicality.

Testing on Windows System

Switching back to Windows brought its own challenges. While the script initially worked, the output appeared in Mandarin due to a team member's system language settings, causing significant confusion and parsing issues. To address this, we adapted our regular expressions to handle both Mandarin and English outputs, a process that required extensive testing and adjustments to achieve consistent results.

This experience underscored the importance of considering diverse user environments in software development. Robust code must account for variations in system configurations to ensure reliability across different setups.

One example of this was adjusting the regex to extract the SSID from the output. Originally, we used a pattern like this:

```python
ssid_pattern = re.compile(r'SSID\s+\d+\s*:\s*(.*)')
```

Figure 12: Original pattern used in the Python tool.

This worked fine when the output was in English but completely failed in Mandarin. After some trial and error, we modified it to accommodate both languages:

```python
ssid_pattern = re.compile(r'SSID\s+\d+\s*:\s*(.*)|网络名称\s+\d+\s*:\s*(.*)')
```

Figure 13: Adjusted the tool to be compatible with different languages.

This made a huge difference, but it was a long process of testing and tweaking until we finally got consistent results.

There was one particular instance where we thought everything was working perfectly—only to realise during a demonstration that some of the data was missing because the regular expressions weren't catching a specific variation in the output format. That was a humbling moment, but it also drove home the importance of thorough testing in a variety of scenarios.

Integration Testing with User Interface

After finalizing the back-end code, we began integration testing with the user interface developed by our teammates. This introduced new complexities, particularly in presenting data to the user. The initial integration was problematic—data displayed incorrectly, and some features failed to communicate with the front end. Resolving these issues required multiple testing rounds, collaborative debugging, and, in some cases, restructuring back-end calculations for better visualization.

A major challenge was how data was passed between the back end and front end. Initially, we used a dictionary, but the front end required a structured JSON format for proper handling. Here’s an example of how we restructured the data:

```python
# Original dictionary data structure
network_data = {
'SSID': ssid,
'Signal Strength': signal_strength,
'Channel': channel
}

# Modified JSON structure for better integration
import json

network_data = {
'networks': [
{
'SSID': ssid,
'SignalStrength': signal_strength,
'Channel': channel
}
]
}
json_data = json.dumps(network_data)
```

Figure 14: Structured JSON format for the proper handling of passed data.

Switching to a structured JSON format significantly improved data flow to the front end, enhancing the user interface's responsiveness and usability. This change made the interface more intuitive and ensured that data was displayed accurately.

During this phase, we realized the critical importance of good documentation. Lack of clear explanations for our code caused misunderstandings and delays, highlighting how poor documentation can hinder progress in collaborative projects. This taught us to prioritize detailed and accessible documentation to improve team efficiency.

We also performed user acceptance testing (UAT) to simulate various scenarios and assess the tool's reliability. In one test, we intentionally overloaded the network to evaluate performance under stress. While the results were not flawless, they identified areas for improvement and allowed us to systematically address them as a team, reinforcing the value of iterative problem-solving and teamwork.

Contribution Table

Name	Contribution
Alexandra Lester	- Researching and suggesting cyber security aspects to product design - Formatting the layout of the report - Editing and reviewing the report

Marco Zhou	- Product development process section of the report - Editing and formatting the report
Nileththi De Silva	- Product requirement section of the report. - Ui design for the product code visualisation. - Graphics and slides designed for the roadshow.
Modarage Perera	- Video creation - Flyer design for the roadshow
Zijie Liao	- Developed Python scripts for Wi-Fi network analysis, including signal strength extraction and channel load analysis. - Integrated and refined multiple analysis features into a unified tool, including network throughput and WLAN traffic analysis. - Created and integrated a user-friendly interface using Tkinter to ensure ease of use for the Wi-Fi analysis tool. - Conducted testing and debugging across different environments, adjusting code to handle system language differences and inconsistencies. - Provided detailed feedback and improvements during tool testing to ensure the stability and reliability of all features.

Reflections on Delivery and Product

The Linux-Based WiFi Heatmap and Load Analyzer reflects significant effort and teamwork, achieving its primary goals of providing insights into WiFi signal strength and network load while addressing cybersecurity vulnerabilities. However, it is not the final version, as some features and refinements remain incomplete due to technical challenges and time constraints.

Level of Completion

The product fulfills core specifications such as signal strength mapping, trilateration, and basic security monitoring. Features like real-time visualization and advanced threat detection were partially implemented or missing due to Linux configuration issues, hardware limitations, and integration challenges. The roadshow presentation demonstrated the prototype effectively, aligning with stakeholder needs but highlighting areas for further development.

Challenges and Outstanding Tasks

Key obstacles included:

1. **Technical Challenges**: Complex Linux configuration, hardware compatibility, and setting up monitor mode forced a pivot to Windows.
2. **Time Constraints**: Limited time restricted the development of a fully interactive interface and comprehensive threat detection.
3. **Resource Limitations**: Insufficient hardware and software compatibility reduced testing and development scope.

Outstanding tasks include refining the interface, enhancing security features, and adding performance metrics like bandwidth trends and automated threat identification.

Potential for Further Development

Future iterations could focus on:

1. Advanced threat detection (e.g., MAC spoofing, real-time MITM alerts).
2. Mobile application development for greater accessibility.
3. Improved visualization with automated tools and richer graphics.
   The product could evolve into a comprehensive WiFi management suite, supporting advanced analytics, multi-channel data collection, and broader platform compatibility.

Reflection on Feedback

Roadshow feedback emphasized enhancing user experience and platform versatility. Improving the interface, aligning with real-world use cases, and focusing on practical applications were key suggestions.

Conclusion

The product establishes a solid foundation but requires further development to reach its full potential. With refinements, it could become a commercially viable tool for network administrators and cybersecurity professionals, guided by stakeholder feedback and evolving needs.

Video Link

https://drive.google.com/file/d/1YE1pNIyW1bZ8N2If7F_ZxUKvxGCLej9e/view?usp=share_link